Database cheatsheet

The model at a glance, where each part lives in the code, and the questions you're likely to get.

LibExpress

At a glance 9 TABLES · BCNF

Copies, not titles
A loan is for one physical `book_copy`, a weak entity keyed by `(book_id, copy_no)`.

Nothing derived is stored
Copy counts, loan status, payment status and member totals are worked out when read.

Withdraw, don't delete
Lowering a book's copy count sets `withdrawn_at`, so loan history stays.

Rules in transactions
Borrowing limits are checked inside a transaction, with row locks so two requests can't both pass.

Where it lives in the code

CONCEPT	FILE	WHAT YOU'LL SEE
SCHEMA: ENTITIES AND RELATIONSHIPS		
Weak entity `book_copy`	`prisma/schema.prisma:95`	`@@id([bookId, copyNo])`, cascades with its book
Loan → copy (composite FK)	`prisma/schema.prisma:130`	`references: [bookId, copyNo]`
writes (M:N) → `book_author`	`prisma/schema.prisma:105`	`@@id([bookId, authorId])`
incurs (1:1) loan ↔ fine	`1_normalize/migration.sql:53`	unique index `fine_loan_id_key` on `fine(loan_id)`
One open loan per copy	`2_book_copies/migration.sql:50`	partial unique `WHERE return_date IS NULL`
Author name used once	`3_author_name_unique/…sql:4`	unique on lowercased first + last name
DERIVED ATTRIBUTES		
total / available copies	`src/server/lib/derived.ts:54`	copies not withdrawn, minus those on an open loan
Loan status	`src/server/lib/derived.ts:14`	`RETURNED` · `OVERDUE` once due date passes · `ACTIVE`
Unpaid fines total	`src/server/lib/derived.ts:77`	sum of `amount` where `paidAt` is null
RULES THE API ENFORCES		
Suspended · > 100.00 owed · 5 loans	`src/server/api/routers/loan.ts:156`	checks in `loan.create` at 156 · 161 · 170
Lend the lowest free copy	`src/server/api/routers/loan.ts:177`	`loans: { none: OPEN_LOAN }`, by `copyNo`
Fine on a late return	`src/server/api/routers/loan.ts:220`	`daysLate × FINE_PER_DAY` (20 per day)
Withdraw copies	`src/server/api/routers/book.ts:42`	`setCopyCount`: withdraws shelf copies, highest first
Row locks	`src/server/lib/db.ts:7`	`lockRow`: `SELECT 1 … FOR UPDATE`
Duplicate → CONFLICT	`src/server/lib/db.ts:37`	`onDuplicate`: Prisma `P2002` → `CONFLICT`
Delete guards	`routers/author.ts:87 · book.ts:186`	author with books, or book with loans, can't be deleted

HTTP endpoints TRPC · NEXT.JS ROUTES

Each procedure is served at `/api/trpc/<router>.<procedure>`: queries as **GET**, mutations as **POST**, batched by the client. **Staff** ones need a signed-in admin, or answer `UNAUTHORIZED`.

METHOD	ENDPOINT	ACCESS	WHAT IT DOES
BOOK `src/server/api/routers/book.ts`			
GET	`book.getAll`	Public	Every book with its authors and its total and available copies.
GET	`book.getById`	Public	One book, with the same details.
POST	`book.create`	Staff	Adds a book and copies 1..n. Its authors must already exist.
POST	`book.update`	Staff	Edits a book. A new copy count adds copies or withdraws shelf copies.
POST	`book.delete`	Staff	Deletes a book that has never been lent.
AUTHOR `src/server/api/routers/author.ts`			
GET	`author.getAll`	Public	Every author, for the book form's picker.
GET	`author.getById`	Public	One author.
POST	`author.create`	Staff	Adds an author. The name must be new, ignoring case.
POST	`author.update`	Staff	Renames an author.
POST	`author.delete`	Staff	Deletes an author who has no books left.
MEMBER `src/server/api/routers/member.ts`			
GET	`member.getAll`	Staff	Every member with their open loans and unpaid fines total.
POST	`member.create`	Staff	Adds a member. A taken email → `CONFLICT`.
POST	`member.update`	Staff	Edits a member, including suspending them.
POST	`member.delete`	Staff	Deletes a member with no loan history.
LOAN `src/server/api/routers/loan.ts`			
GET	`loan.getOpen`	Staff	Books still out, with the fine owed if returned now.
GET	`loan.getTransactions`	Staff	Checkouts, returns, fines and payments, newest first, optionally `since` a date.
GET	`loan.getHistory`	Staff	Every loan one member has had, with any fine.
POST	`loan.create`	Staff	Checks the borrowing rules and lends the lowest free copy.
POST	`loan.return`	Staff	Stamps the return time and charges a fine if it's late.
FINE & DASHBOARD `fine.ts, dashboard.ts`			
POST	`fine.update`	Staff	Marks a fine paid or unpaid.
GET	`dashboard.get`	Staff	Totals, circulation by month, top books, late books and the activity feed.
OTHER ROUTES `src/app/api`			
GET POST	`/api/auth/*`	Public	NextAuth sign-in, sign-out and session.
POST	`/api/relay/upload`	Staff	Uploads a cover image to the relay server and returns its URL.
OUTGOING CALLS `src/server/lib/relay.ts`			
POST	`RELAY_URL/api/files/upload`	API key	Sends the image file to the relay image server.
GET	`RELAY_URL/api/files/download/:id`	—	Gets the public URL of the uploaded image.

Likely questions

Why is `book_copy` a weak entity?

`copy_no` restarts at 1 for every book, so it only identifies a copy together with `book_id`. A copy can't exist without its book.

Why is a loan an entity, not a relationship?

The same member can borrow the same copy many times, and a fine belongs to one specific loan, so each loan needs its own id.

What is the cardinality of `incurs`?

One-to-one. A fine has exactly one loan (1..1, total); a loan has at most one fine (0..1). The unique `loan_id` enforces it.

Why not store `total_copies` or loan status?

They depend on other rows or on today's date, not on the key alone. Storing them could go stale, so they're computed when read.

Then why is `fine.amount` stored?

It's what was charged at the rate in force that day. Changing the rate later mustn't rewrite old fines.

Is the schema in BCNF?

Yes. In every table, the left side of each non-trivial FD is a candidate key: `id`, or an alternate key like `isbn`, `email` or `loan_id`.

Why doesn't `fine` link to `member`?

The member is reachable through the loan. A second link would repeat that fact and could disagree with it.

What stops two staff lending the last copy?

`loan.create` locks the book row (`FOR UPDATE`). If anything slips past, the partial unique index rejects a second open loan on that copy.

Why no FK on `activity_log.entity_id`?

It's a historical record. The row it names may have been deleted since, and the log entry must survive that.

What happens when the copy count goes down?

Copies on the shelf are withdrawn, highest number first. If not enough are on the shelf, the update fails.

Why raw SQL in `lockRow`?

Prisma has no `SELECT … FOR UPDATE`. Everything else goes through Prisma's query API.

Why `RESTRICT` on `activity_log.admin_id`?

The log no longer keeps a copy of the admin's email, so deleting the admin would lose who made the change.